Travaux dirigés — Langage C

TD — Stockage de matrices en C

Programmation modulaire et séparée, fichiers texte et allocation dynamique

A. ADELL

Public	Élèves de 3e année STI
Auteur	A. ADELL
Durée indicative	2 séances de 1 h 30 (+ travail personnel)
Prérequis	pointeurs, `malloc`/`free`, structures, `fopen`/`fprintf`/`fscanf`
Produit final	une petite bibliothèque de 4 modules, un programme de démonstration et ses fichiers de test
Contenu	sujet (parties 1 à 6)

Année universitaire 2026–2027

Sommaire

1 Comprendre le problème **3**

2 Concevoir la bibliothèque **5**

3 Exercice 1 — Allocation et destruction **13**

4 Exercice 2 — Entrées-sorties **19**

5 Exercice 3 — Calcul matriciel **25**

6 Exercice 4 — Démonstration complète **31**

Comment travailler avec ce sujet

- Les parties 1 et 2 posent le problème et *fixent les interfaces* : lisez-les entièrement avant d'écrire du code. Les prototypes de la partie 2 sont imposés.
- Les exercices 1 à 4 se font dans l'ordre : chaque module s'appuie sur les précédents.
- Chaque exercice se termine par des *questions de réflexion* (à préparer par écrit) et des *critères de validation* : un exercice n'est terminé que lorsque toutes les cases peuvent être cochées.
- Les encadrés **Indice** sont progressifs : ne lisez l'indice suivant que si vous êtes bloqué.

1 Comprendre le problème

1.1 Contexte

On souhaite écrire une petite *bibliothèque* C permettant de **créer**, **sauvegarder**, **relire**, **afficher** et **calculer** (somme et produit) des matrices d'entiers. Une bibliothèque est un ensemble de fonctions réutilisables qui ne contient pas de fonction `main` : elle est destinée à être utilisée par un ou plusieurs programmes.

Le code sera découpé en *modules* (voir partie 2), chacun regroupant des fonctions d'un même thème : gestion de la mémoire, entrées-sorties, calcul. Un programme de démonstration, écrit dans un fichier séparé, utilisera ces modules.

1.2 Objectifs pédagogiques

À l'issue du TD, vous saurez :

1. séparer *l'interface* d'un module (fichier `.h`) de son *implémentation* (fichier `.c`) ;
2. compiler séparément plusieurs modules puis les assembler par *édition de liens* ;
3. représenter une matrice de taille variable en mémoire et l'allouer dynamiquement, y compris en cas d'échec partiel ;
4. établir et respecter des règles de *propriété de la mémoire* (qui alloue, qui utilise, qui libère) pour éviter toute fuite ;
5. lire et écrire des données dans un fichier texte en vérifiant chaque opération ;
6. programmer l'addition et la multiplication de matrices en contrôlant les dimensions ;
7. concevoir une gestion d'erreurs uniforme et la vérifier par des tests.

1.3 Vocabulaire

Terme	Définition
Module	Couple de fichiers `nom.h` / `nom.c` regroupant des fonctions d'un même thème.
Interface (fichier d'en-tête)	Fichier `.h` : ce que le module *offre* (types, constantes, prototypes). C'est le contrat avec les utilisateurs du module.
Implémentation	Fichier `.c` : *comment* le module réalise ce qu'il offre (définitions des fonctions).
Compilation séparée	Chaque `.c` est compilé indépendamment en un fichier objet `.o` (`gcc -c`).
Édition de liens	Assemblage des fichiers objets en un exécutable ; chaque appel de fonction est relié à sa définition.
Flux (`FILE *`)	Objet de la bibliothèque standard représentant un fichier ouvert (ou l'écran, le clavier).
Propriétaire d'une zone mémoire	Code chargé de libérer cette zone, exactement une fois.
Fuite mémoire	Zone allouée qui n'est jamais libérée.

1.4 Les matrices

Une *matrice* de dimensions $n \times p$ est un tableau de nombres à n lignes et p colonnes ($n, p \geq 1$). On note a_{ij} le coefficient situé ligne i, colonne j. Par exemple, la matrice suivante est de dimensions 4×3 :

$$A = \begin{pmatrix} 25 & 48 & 84 \\ 15 & 33 & 11 \\ 45 & 88 & 10 \\ 0 & 12 & -5 \end{pmatrix} \qquad \text{4 lignes, 3 colonnes ;} \quad a_{43} = -5.$$

> **Indices mathématiques et indices C**
>
> En mathématiques, les indices commencent à 1 ; en C, ils commencent à 0. Le coefficient a_{43} ci-dessus est donc `A.coef[3][2]` dans le programme. Toutes les boucles du TD vont de `0` à `nb_lignes - 1` (ou `nb_colonnes - 1`).

1.5 Le format du fichier texte

Une matrice est sauvegardée dans un fichier texte (lisible et modifiable avec un éditeur) selon le format suivant :

- **première ligne** : deux entiers, le nombre de lignes n puis le nombre de colonnes p ;
- **lignes suivantes** : n lignes contenant chacune les p coefficients d'une ligne de la matrice, séparés par des espaces.

```
4 3
25 48 84
15 33 11
45 88 10
0 12 -5
```

Fichier `A.txt` représentant la matrice A. La ligne `4 3` annonce 4 lignes de 3 coefficients ; on lit ensuite exactement $4 \times 3 = 12$ entiers.

Deux règles précisent ce format, afin que l'écriture et la lecture soient parfaitement cohérentes :

1. **En écriture** (fonction `mat_ecrire`), le format est *canonique* : un seul espace entre deux coefficients, aucun espace en fin de ligne, un saut de ligne après chaque ligne (y compris la dernière). Le fichier ci-dessus est exactement celui que produit `mat_ecrire`.
2. **En lecture** (fonction `mat_lire`), on est *tolérant sur les blancs* : un nombre quelconque d'espaces, de tabulations ou de sauts de ligne peut séparer deux entiers. En revanche, on est *strict sur le contenu* : dimensions entre 1 et `MAT_DIM_MAX`, exactement $n \times p$ coefficients, chacun étant un entier représentable par un `int`.

> **Piège : le signe moins**
>
> Le signe « moins » d'un fichier doit être le caractère ASCII `-` (trait d'union). Un texte copié depuis un traitement de texte ou un PDF contient souvent le caractère typographique « − » (U+2212), qui n'est pas un chiffre ni un signe pour `strtol` ou `fscanf` : le fichier est alors *mal formé*.

1.6 Vue d'ensemble : du fichier au résultat

La figure 1 montre le trajet des données dans le programme final : deux matrices sont lues depuis des fichiers, stockées en mémoire, combinées, puis le résultat est affiché et enregistré.

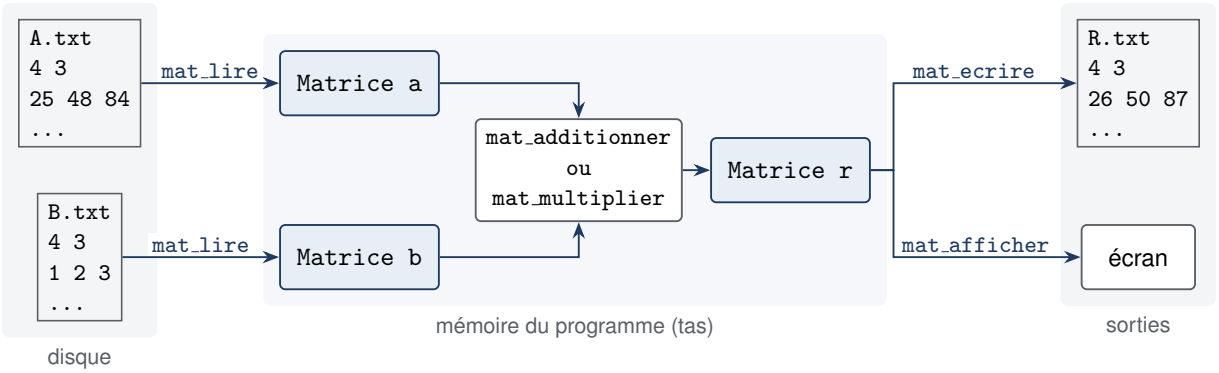


Figure 1: Circulation des données : lecture, calcul, affichage et enregistrement.

1.7 Ce que vous devez obtenir

Vous ne partez pas d'une page blanche : on vous fournit un dossier de départ (section 2.10) qui contient les en-têtes complets, des fichiers `.c` déjà commencés, des programmes de test et des fichiers de données. À la fin du TD, ce dossier contiendra :

- les 9 fichiers sources de la bibliothèque et du programme, tous complétés (`matrice.h`, `matrice.c`, `matrice_alloc.h`, `matrice_alloc.c`, `matrice_io.h`, `matrice_io.c`, `matrice_calcul.h`, `matrice_calcul.c`, `main.c`) ;
- les trois programmes de test fournis (`test_ex1.c`, `test_ex2.c`, `test_ex3.c`), qui affichent tous `[OK]` ;
- un programme `demo_matrices` qui compile sans aucun avertissement avec `-Wall -Wextra -Wpedantic` et ne présente aucune fuite mémoire sous `valgrind`, y compris dans les cas d'erreur.

2 Concevoir la bibliothèque

Cette partie fixe toutes les décisions de conception : découpage en modules, représentation des données, prototypes et gestion des erreurs. Ces choix sont **imposés** : ils garantissent que vos modules pourront être combinés avec ceux de vos camarades ou avec la correction.

2.1 Pourquoi découper en modules ?

Tout écrire dans un seul fichier `main.c` fonctionnerait, mais le programme deviendrait vite difficile à lire, à tester et à réutiliser. Le découpage en modules permet :

- de **réutiliser** la bibliothèque dans plusieurs programmes (ici : `main.c` pour la démonstration et les programmes `test_ex1.c`, `test_ex2.c`, `test_ex3.c` pour les tests, sans copier une seule ligne) ;
- de **cacher les détails** : un utilisateur du module n'a besoin de lire que le `.h` ;

- de **recompiler seulement ce qui a changé**.

	Fichier d'en-tête `.h`	**Implémentation** `.c`	**Programme principal**
Contient	types, constantes, prototypes commentés (le contrat)	définitions des fonctions, fonctions internes `static`	la fonction `main` et la logique de l'application
Inclus par	tout fichier qui utilise le module	— (jamais inclus)	—
Compilé	jamais seul (il est recopié par `#include`)	`gcc -c module.c` → `module.o`	`gcc -c main.c` → `main.o`

2.2 Organisation du projet

```
td_matrices/
|-- matrice.h
|-- matrice.c
|-- matrice_alloc.h
|-- matrice_alloc.c
|-- matrice_io.h
|-- matrice_io.c
|-- matrice_calcul.h
|-- matrice_calcul.c
|-- main.c
|-- test_ex1.c
|-- test_ex2.c
|-- test_ex3.c
`-- donnees/
    |-- A.txt
    `-- ...
```

Module	**Rôle**
`matrice`	type `Matrice`, codes d'erreur, outils communs
`matrice_alloc`	créer et détruire une matrice
`matrice_io`	lire, écrire, afficher une matrice
`matrice_calcul`	additionner, multiplier
`main.c`	démonstration (lit, calcule, enregistre)
`test_ex`*N*`.c`	tests de l'exercice *N* (fournis)

Tous les modules de la bibliothèque portent le préfixe `matrice_` et toutes leurs fonctions publiques le préfixe `mat_` : on sait ainsi immédiatement d'où vient un nom.

Pourquoi un fichier matrice.c ?

`matrice.h` déclare deux fonctions utilisées par tous les modules (`mat_message_erreur` et `mat_est_valide`). Une fonction doit être définie *une seule fois* dans tout le programme : ces deux définitions sont donc placées dans `matrice.c`, et non dans un en-tête ni dans l'un des autres modules.

2.3 Dépendances entre modules

Une flèche $X \to Y$ signifie « X contient `#include "Y"` ». Le graphe ne doit pas contenir de cycle, et un module n'inclut que ce dont il a réellement besoin : `matrice_calcul` n'a rien à faire des fichiers, il n'inclut donc pas `matrice_io.h`.

Bonnes pratiques pour les fichiers d'en-tête

- **Garde contre l'inclusion multiple.** Chaque `.h` est entouré de `#ifndef MATRICE_IO_H` / `#define MATRICE_IO_H` / `#endif`. Sans cela, `main.c`, qui inclut `matrice.h` directement *et* via les autres en-têtes, définirait deux fois le type `Matrice` : erreur de compilation.
- **Chaque `.c` inclut son propre `.h`** : le compilateur vérifie alors que prototypes et définitions concordent.
- **Un `.h` inclut ce dont il a besoin** : `matrice_io.h` utilise `FILE`, il inclut donc `<stdio.h>`.

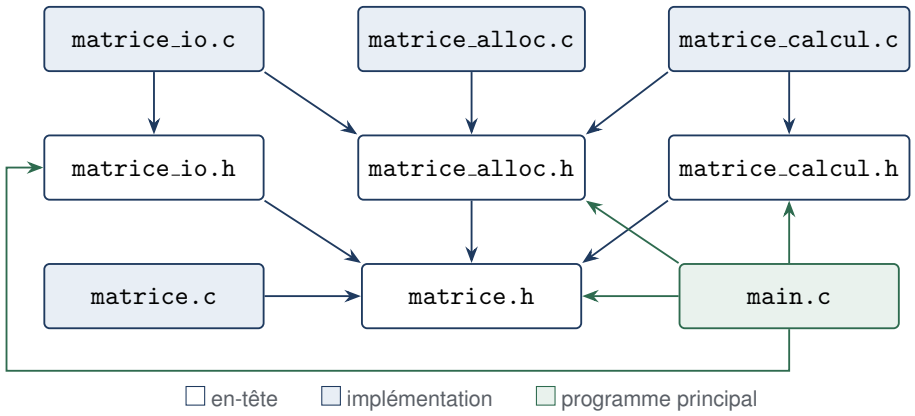


Figure 2: Graphe des inclusions. Les programmes de test `test_exN.c` incluent les en-têtes de la même façon que `main.c`.

- **Les fonctions internes sont** `static` : elles sont invisibles hors de leur fichier et n'apparaissent pas dans le `.h`.
- **Aucune définition de fonction dans un** `.h`, et aucune fonction définie dans deux modules.

2.4 Compilation séparée

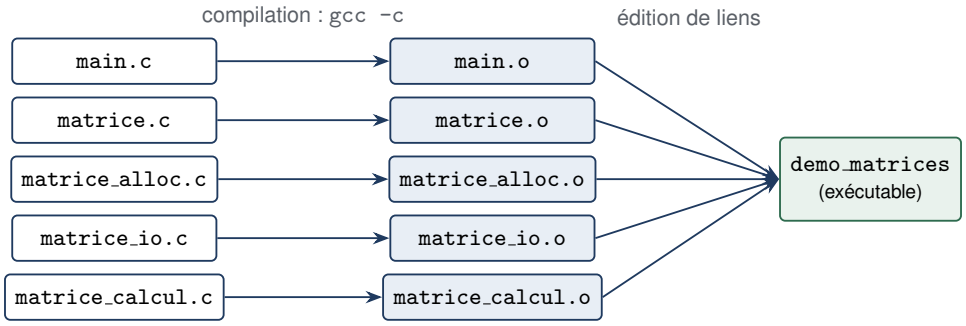


Figure 3: Chaque `.c` donne un fichier objet ; l'éditeur de liens les assemble. Les `.h` ne sont jamais compilés seuls.

```
# 1) Compilation séparée : un fichier objet par fichier source
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c matrice.c
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c matrice_alloc.c
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c matrice_io.c
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c matrice_calcul.c
gcc -std=c11 -Wall -Wextra -Wpedantic -g -c main.c
# 2) Édition de liens
gcc main.o matrice.o matrice_alloc.o matrice_io.o matrice_calcul.o \
    -o demo_matrices
```

Si une fonction est déclarée mais définie nulle part, la compilation réussit mais l'édition de liens échoue (*undefined reference*) ; si elle est définie dans deux modules, l'édition de liens échoue aussi (*multiple definition*).

2.5 Représentation d'une matrice en mémoire

Une matrice est représentée par la structure suivante, définie dans `matrice.h` :

```c
typedef struct matrice {
    int **coef;       /* coef[i][j] : ligne i, colonne j ; NULL si vide */
    int nb_lignes;    /* 0 si vide, sinon entre 1 et MAT_DIM_MAX        */
    int nb_colonnes;  /* 0 si vide, sinon entre 1 et MAT_DIM_MAX        */
} Matrice;

#define MAT_DIM_MAX  10000
#define MATRICE_VIDE ((Matrice){ NULL, 0, 0 })
```

- `coef` donne accès aux coefficients ; `nb_lignes` et `nb_colonnes` mémorisent les dimensions, indispensables puisqu'un pointeur ne connaît pas la taille de la zone qu'il désigne.
- Le `typedef` permet d'écrire `Matrice` au lieu de `struct matrice` (les deux restent valides).
- `MAT_DIM_MAX` borne les dimensions acceptées, pour se protéger des fichiers aberrants.
- `MATRICE_VIDE` désigne une matrice qui ne possède aucune mémoire. C'est l'**état initial** de toute variable `Matrice` et l'état laissé par la destruction.

Un tableau de pointeurs de lignes. `coef` pointe vers un tableau de `nb_lignes` pointeurs ; le pointeur `coef[i]` pointe vers un tableau de `nb_colonnes` entiers, la ligne i. On écrit ainsi `a.coef[i][j]`, comme pour un tableau à deux dimensions, mais ce *n'est pas* un tableau bidimensionnel C (`int t[4][3]`) : les lignes sont des blocs distincts, placés n'importe où dans le tas (figure 4).

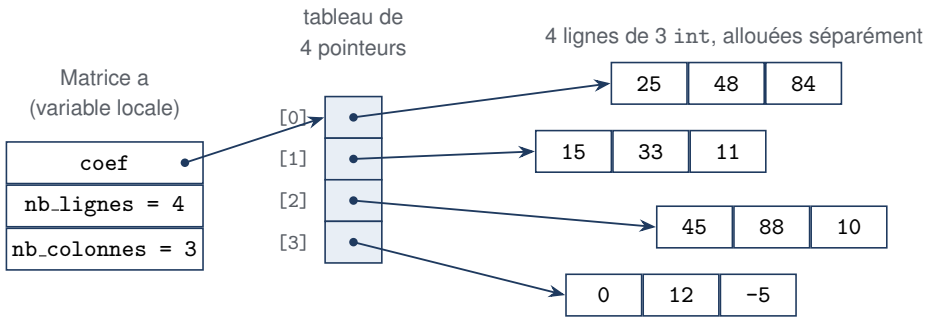


Figure 4: Représentation de la matrice A : $1 + n$ blocs alloués. Les lignes ne sont pas forcément contiguës en mémoire.

Une autre représentation possible. On pourrait stocker les $n \times p$ coefficients dans un seul bloc contigu (`int *coef`) et accéder au coefficient (i, j) par `coef[i * nb_colonnes + j]`.

	Tableau de lignes (`int **`)	**Bloc contigu** (`int *`)
Accès	`coef[i][j]`, naturel	`coef[i*p + j]`, calcul d'indice
Allocations	$1 + n$ appels, échec partiel à gérer	1 seul appel
Libération	$1 + n$ appels, dans le bon ordre	1 seul appel
Performances	lignes dispersées, moins bon usage du cache	meilleur usage du cache
Intérêt pour le TD	entraîne à l'allocation à deux niveaux	plus simple, moins formateur

Choix retenu. Nous utilisons la représentation `int **` : elle offre la notation `coef[i][j]` et, surtout, oblige à traiter correctement l'échec partiel d'une allocation à deux niveaux, objectif central du TD. Elle est utilisée de façon identique dans tous les modules.

2.6 Pourquoi ne pas renvoyer directement une matrice ?

Une première idée, naturelle, serait d'écrire des fonctions qui renvoient ou reçoivent des matrices *par valeur* :

```
Matrice lire(FILE *f);                    /* conception naïve */
void    ecrire(FILE *f, Matrice m);
Matrice additionner(Matrice a, Matrice b);
```

Cette conception pose trois problèmes.

1. **Impossible de signaler une erreur proprement.** Une fonction qui renvoie une `Matrice` ne peut pas dire *pourquoi* elle a échoué (fichier mal formé ? dimensions incompatibles ? mémoire épuisée ?), et l'appelant risque d'utiliser une matrice invalide sans s'en rendre compte. Une fonction `ecrire` de type `void` ne peut même pas signaler qu'elle a échoué.
2. **Copie superficielle.** Passer une `Matrice` par valeur copie le *pointeur* `coef`, pas les coefficients : l'appelant et la fonction partagent la même mémoire. Rien n'empêche alors la fonction de modifier, voire de libérer, les données de l'appelant.
3. **Responsabilités floues.** Qui libère le résultat ? Que vaut-il en cas d'échec ? Rien dans le prototype ne le dit.

2.7 Stratégie de gestion des erreurs

Nous adoptons une stratégie unique pour tout le projet.

- **Toute fonction qui peut échouer renvoie un code** de type `MatErreur`, qui vaut `MAT_OK` (zéro) en cas de succès.
- **Le résultat passe par un paramètre de sortie** (`Matrice *res`).
- **Les matrices d'entrée sont passées par pointeur** `const` : pas de copie, et la fonction s'engage à ne pas les modifier.
- **Les fonctions de la bibliothèque n'affichent rien** en cas d'erreur : elles renvoient un code, et c'est le programme appelant qui décide quoi afficher (grâce à `mat_message_erreur`).

```
typedef enum {
    MAT_OK = 0,           /* succès                                  */
    MAT_ERR_ARGUMENT,     /* paramètre invalide (pointeur NULL, ...)  */
    MAT_ERR_ALLOCATION,   /* échec de malloc/calloc                  */
    MAT_ERR_LECTURE,      /* erreur du flux pendant la lecture       */
    MAT_ERR_FORMAT,       /* contenu du fichier mal formé ou tronqué  */
    MAT_ERR_ECRITURE,     /* erreur du flux pendant l'écriture       */
    MAT_ERR_DIMENSIONS,   /* dimensions incompatibles                */
    MAT_ERR_DEPASSEMENT   /* résultat hors de l'intervalle des int    */
} MatErreur;
```

2.8 Règles de propriété de la mémoire

Pour qu'aucune zone ne soit oubliée ni libérée deux fois, toutes les fonctions respectent les règles suivantes. Elles seront citées dans les exercices sous la forme **R1**, **R2**, etc.

R1 Toute variable `Matrice` est initialisée à `MATRICE_VIDE` dès sa déclaration.

R2 Une fonction qui réussit (`MAT_OK`) et remplit `*res` *transfère la propriété* du résultat à l'appelant, qui devra appeler `mat_detruire(&res)`.

R3 Une fonction qui échoue laisse `*res` égal à `MATRICE_VIDE` et a libéré tout ce qu'elle avait alloué : l'appelant n'a rien à libérer.

R4 Une fonction ne modifie et ne libère jamais une matrice reçue en entrée (`const Matrice *`).

R5 `*res` est un paramètre de sortie : son ancien contenu est écrasé. S'il possédait de la mémoire, l'appelant doit l'avoir détruite avant l'appel. `res` doit être distinct des entrées.

R6 `mat_detruire` remet la matrice à `MATRICE_VIDE` : un second appel est sans effet.

R7 Un fichier est fermé par le code qui l'a ouvert. Les fonctions de `matrice_io` reçoivent un flux déjà ouvert et ne le ferment jamais.

2.9 Prototypes définitifs

```c
/* matrice.h -- outils communs */
const char *mat_message_erreur(MatErreur code);
int         mat_est_valide(const Matrice *m);

/* matrice_alloc.h */
MatErreur mat_allouer(Matrice *res, int nb_lignes, int nb_colonnes);
void      mat_detruire(Matrice *m);

/* matrice_io.h */
MatErreur mat_lire(FILE *f, Matrice *res);
MatErreur mat_ecrire(FILE *f, const Matrice *m);
void      mat_afficher(const char *nom, const Matrice *m);

/* matrice_calcul.h */
MatErreur mat_additionner(const Matrice *a, const Matrice *b,
                          Matrice *res);
MatErreur mat_multiplier(const Matrice *a, const Matrice *b,
                         Matrice *res);
```

Fonction	Rôle	En cas de succès	Erreurs possibles
`mat_message_erreur`	message lisible associé à un code	chaîne constante	—
`mat_est_valide`	`m` est-elle une matrice utilisable ?	1 (sinon 0)	—
`mat_allouer`	crée une matrice $n \times p$ remplie de 0	`*res` possède sa mémoire	ARGUMENT, ALLOCATION
`mat_detruire`	libère la matrice, la remet à vide	—	— (sans effet si vide ou `NULL`)
`mat_lire`	lit une matrice depuis un flux ouvert	`*res` possède sa mémoire	ARGUMENT, FORMAT, LECTURE, ALLOCATION
`mat_ecrire`	écrit une matrice au format canonique	fichier écrit	ARGUMENT, ECRITURE
`mat_afficher`	affiche à l'écran (mise au point)	—	— (signale une matrice vide)
`mat_additionner`	res $= a + b$	`*res` possède sa mémoire	ARGUMENT, DIMENSIONS, ALLOCATION, DEPASSEMENT
`mat_multiplier`	res $= a \times b$	`*res` possède sa mémoire	ARGUMENT, DIMENSIONS, ALLOCATION, DEPASSEMENT

Dans la dernière colonne, les codes sont abrégés : ARGUMENT signifie `MAT_ERR_ARGUMENT`, etc.

2.10 Fichiers fournis et mise en route

On vous fournit un dossier de départ. Les en-têtes sont *complets* : les prototypes sont déjà écrits, vous n'avez pas à les modifier. Les fichiers `.c` sont *commencés* : vous les complétez étape par étape en suivant les exercices.

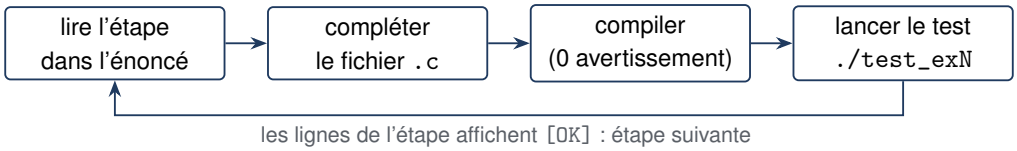
Fichier	État	Utilisé dans
`matrice.h, matrice_alloc.h`	complet, ne pas modifier	tous les exercices
`matrice_io.h, matrice_calcul.h`	complet, ne pas modifier	exercices 2 à 4
`test_ex1.c, test_ex2.c, test_ex3.c`	complet, ne pas modifier	tests des exercices 1, 2 et 3
`donnees/*.txt`	complet	exercices 2 et 4
`matrice.c, matrice_alloc.c`	**à compléter**	exercice 1
`matrice_io.c`	**à compléter**	exercice 2
`matrice_calcul.c`	**à compléter**	exercice 3
`main.c`	**à compléter**	exercice 4

Les repères dans les fichiers à compléter.

- `FOURNI` : code déjà écrit et correct. Lisez-le pour comprendre, mais ne le modifiez pas.
- `(1.c) À COMPLÉTER` : emplacement d'une étape. Le numéro renvoie à l'étape du même numéro dans l'énoncé, qui explique quoi écrire. Le commentaire donne souvent le début du code.

- `ligne provisoire` : ligne qui permet au fichier de compiler *avant* que vous l'ayez complété. Le commentaire dit à quelle étape la supprimer ou la remplacer. Par exemple, `(void)m;` « utilise » artificiellement le paramètre `m` pour éviter l'avertissement *unused parameter* tant que vous ne l'utilisez pas vraiment.

Méthode de travail. Travaillez par petites étapes et vérifiez chacune avant de passer à la suivante :



Premier essai, avant d'écrire la moindre ligne.

1. Copiez le dossier fourni dans votre espace de travail, puis ouvrez un terminal *dans ce dossier* (commande `cd`).
2. Compilez le premier programme de test :

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g -DMAT_TEST_ALLOC \
    test_ex1.c matrice.c matrice_alloc.c -o test_ex1
```

3. La compilation doit réussir *sans aucun message*. Lancez alors `./test_ex1`. Vous obtenez :

```
=== Exercice 1 : matrice.c ===
[ECHEC] (1.a) message pour MAT_ERR_DIMENSIONS
[ECHEC] (1.a) message pour MAT_ERR_DEPASSEMENT
[ECHEC] (1.b) une matrice 1 x 2 est valide
[OK]    (1.b) NULL n'est pas valide
[OK]    (1.b) MATRICE_VIDE n'est pas valide
[OK]    (1.b) une matrice de 0 ligne n'est pas valide
=== Exercice 1 : matrice_alloc.c ===
[ECHEC] (1.c) mat_allouer refuse 0 ligne
[ECHEC] (1.c) mat_allouer refuse -2 colonnes
[ECHEC] (1.c) mat_allouer refuse une dimension > MAT_DIM_MAX
[ECHEC] (1.e) mat_allouer(&m, 3, 4) renvoie MAT_OK
[ECHEC] (1.e) m a 3 lignes et 4 colonnes
[ECHEC] (1.d) tous les coefficients valent 0
[OK]    (1.f) apres mat_detruire, m est vide
[OK]    (1.f) un second mat_detruire est sans danger
[OK]    (1.f) mat_detruire(NULL) ne plante pas
=== Exercice 1 : echecs d'allocation simules ===
[ECHEC] (1.d) chaque echec renvoie MAT_ERR_ALLOCATION et laisse *res vide

Bilan : 6 / 16 verifications reussies.
```

C'est normal : les fonctions ne sont pas encore écrites. Les quelques `[OK]` correspondent à des cas que le code provisoire traite déjà par hasard. Votre objectif dans l'exercice 1 est d'obtenir `Bilan : 16 / 16`.

Lire un message du compilateur

Un message comme `matrice_alloc.c:52:13: error: 'coef' undeclared` se lit : *fichier* `matrice_alloc.c`, *ligne* 52, *colonne* 13, puis la description du problème. Corrigez toujours le

> *premier* message d'abord : les suivants en sont souvent la conséquence. Un `warning` n'empêche pas la compilation, mais il signale presque toujours une erreur : dans ce TD, on n'en accepte aucun.

3 Exercice 1 — Allocation et destruction

Fichiers à compléter : `matrice.c` et `matrice_alloc.c`. Objectif : créer une matrice de dimensions quelconques sur le tas et la libérer sans fuite, y compris lorsqu'une allocation échoue en cours de route.

3.1 Ce qu'il faut comprendre

Les dimensions d'une matrice ne sont connues qu'*à l'exécution* (par exemple en lisant la première ligne d'un fichier). Un tableau `int t[4][3]` a une taille fixée à l'écriture du programme : il ne convient pas. On réserve donc la mémoire sur le *tas* pendant l'exécution, avec `calloc`, et on la rend avec `free`.

Avec la représentation choisie (figure 4), une matrice $n \times p$ demande $1 + n$ réservations :

1. un tableau de n pointeurs, `coef`, un par ligne ;
2. puis, pour chaque ligne i, un tableau de p entiers, `coef[i]`.

La destruction rend ces $1 + n$ zones dans l'ordre *inverse* : d'abord les lignes, puis le tableau qui contient leurs adresses.

> **Rappel : calloc**
>
> `calloc(nb, taille)` réserve `nb` éléments de `taille` octets, *tous mis à zéro*, et renvoie leur adresse, ou `NULL` s'il n'y a plus assez de mémoire. Il faut *toujours* tester ce résultat. Dans ce TD, on utilise `mat_calloc`, qui s'emploie exactement comme `calloc` mais permet aux tests de simuler un manque de mémoire.

3.2 Les modules fournis

Les deux en-têtes sont complets : lisez-les attentivement, les commentaires décrivent précisément ce que chaque fonction doit faire.

Fichier fourni `matrice_alloc.h`

```c
/*
 * matrice_alloc.h -- Création et destruction des matrices.
 */
#ifndef MATRICE_ALLOC_H
#define MATRICE_ALLOC_H

#include "matrice.h"

/*
 * Alloue une matrice nb_lignes x nb_colonnes dont tous les coefficients
 * valent 0.
 *   Préconditions : res != NULL ; 1 <= nb_lignes, nb_colonnes <= MAT_DIM_MAX.
 *   Succès (MAT_OK) : *res possède sa propre mémoire ; l'appelant devra
 *                     appeler mat_detruire(res).
 *   Échec           : *res == MATRICE_VIDE (si res != NULL), aucune fuite.
```

```c
 *   Erreurs          : MAT_ERR_ARGUMENT, MAT_ERR_ALLOCATION.
 */
MatErreur mat_allouer(Matrice *res, int nb_lignes, int nb_colonnes);

/*
 * Libère toute la mémoire possédée par *m puis remet *m à MATRICE_VIDE.
 * Sans effet si m == NULL ou si *m est déjà vide : un second appel est
 * donc sans danger (pas de double libération).
 */
void mat_detruire(Matrice *m);

#ifdef MAT_TEST_ALLOC
/* Instrumentation réservée aux tests (compiler avec -DMAT_TEST_ALLOC) :
 * nombre d'allocations qui réussiront encore avant un échec simulé ;
 * -1 (valeur par défaut) désactive la simulation. */
extern int mat_test_allocs_autorisees;
#endif

#endif /* MATRICE_ALLOC_H */
```

Le type `Matrice`, la constante `MATRICE_VIDE` et les codes d'erreur sont dans `matrice.h` (présentés en partie 2). Voici les deux fichiers `.c` de départ :

Fichier fourni `matrice.c`

```c
/*
 * matrice.c -- Fonctions utilitaires communes à tous les modules.
 *
 * FICHIER À COMPLÉTER (exercice 1, étapes 1.a et 1.b).
 */
#include <stddef.h>
#include "matrice.h"

const char *mat_message_erreur(MatErreur code)
{
    switch (code) {
    case MAT_OK:              return "succès";
    case MAT_ERR_ARGUMENT:    return "argument invalide";
    case MAT_ERR_ALLOCATION:  return "mémoire insuffisante";
    case MAT_ERR_LECTURE:     return "erreur de lecture du fichier";
    case MAT_ERR_FORMAT:      return "fichier mal formé";
    case MAT_ERR_ECRITURE:    return "erreur d'écriture du fichier";

    /* (1.a) À COMPLÉTER : les cas MAT_ERR_DIMENSIONS et
     *       MAT_ERR_DEPASSEMENT, sur le même modèle que ci-dessus. */

    default:                  break;
    }
    return "erreur inconnue";
}

int mat_est_valide(const Matrice *m)
{
    /* (1.b) À COMPLÉTER : renvoyer 1 si les QUATRE conditions suivantes
     *       sont vraies, 0 sinon :
     *         - m n'est pas NULL ;
     *         - m->coef n'est pas NULL ;
     *         - 1 <= m->nb_lignes <= MAT_DIM_MAX ;
     *         - 1 <= m->nb_colonnes <= MAT_DIM_MAX.
     *       Une seule instruction suffit :
     *         return m != NULL && m->coef != NULL && ... ;            */

```

```
    (void)m;      /* ligne provisoire : à supprimer en (1.b) */
    return 0;     /* ligne provisoire : à remplacer en (1.b) */
}
```

Fichier fourni `matrice_alloc.c`

```c
/*
 * matrice_alloc.c -- Création et destruction des matrices.
 *
 * FICHIER À COMPLÉTER (exercice 1, étapes 1.c à 1.f).
 *
 * Représentation : coef est un tableau de nb_lignes pointeurs ; chaque
 * coef[i] pointe vers un tableau de nb_colonnes int alloué séparément.
 */
#include <stdlib.h>
#include "matrice_alloc.h"

/* ---------------------------------------------------------------------
 * FOURNI -- Ne pas modifier.
 * mat_calloc s'utilise exactement comme calloc. Compilé avec l'option
 * -DMAT_TEST_ALLOC, il permet aux programmes de test de simuler un
 * manque de mémoire.
 * --------------------------------------------------------------------- */
#ifdef MAT_TEST_ALLOC
int mat_test_allocs_autorisees = -1;

static void *mat_calloc(size_t nb, size_t taille)
{
    if (mat_test_allocs_autorisees == 0)
        return NULL;                        /* échec simulé */
    if (mat_test_allocs_autorisees > 0)
        mat_test_allocs_autorisees--;
    return calloc(nb, taille);
}
#else
#define mat_calloc calloc
#endif

MatErreur mat_allouer(Matrice *res, int nb_lignes, int nb_colonnes)
{
    /* FOURNI : on vérifie res, puis on le met à l'état vide (règle R3). */
    if (res == NULL)
        return MAT_ERR_ARGUMENT;
    *res = MATRICE_VIDE;

    /* (1.c) À COMPLÉTER : si nb_lignes ou nb_colonnes n'est pas compris
     *       entre 1 et MAT_DIM_MAX, renvoyer MAT_ERR_ARGUMENT.          */


    /* FOURNI : tableau des nb_lignes pointeurs de lignes (tous à NULL). */
    int **coef = mat_calloc((size_t)nb_lignes, sizeof *coef);
    if (coef == NULL)
        return MAT_ERR_ALLOCATION;

    /* (1.d) À COMPLÉTER : allouer chaque ligne.
     *   for (int i = 0; i < nb_lignes; i++) {
     *       coef[i] = mat_calloc(..., ...);    // nb_colonnes entiers
     *       if (coef[i] == NULL) {
     *           for (int k = i - 1; k >= 0; k--)  // lignes déjà
     *               free(coef[k]);                // obtenues
     *           free(coef);                       // puis le tableau
     *           return MAT_ERR_ALLOCATION;
```

```c
     *       }
     *   }                                                    */

    (void)nb_colonnes;   /* ligne provisoire : à supprimer en (1.d) */

    /* (1.e) À COMPLÉTER : maintenant que tout a réussi, remplir *res
     *       (trois affectations : coef, nb_lignes, nb_colonnes)
     *       et renvoyer MAT_OK.                                  */

    free(coef);                  /* ligne provisoire : à supprimer en (1.e) */
    return MAT_ERR_LECTURE;      /* ligne provisoire : à remplacer en (1.e) */
}

void mat_detruire(Matrice *m)
{
    if (m == NULL)                 /* FOURNI */
        return;

    /* (1.f) À COMPLÉTER : si m->coef n'est pas NULL :
     *         - libérer chaque ligne m->coef[i], i de 0 à nb_lignes - 1 ;
     *         - PUIS libérer le tableau m->coef lui-même.              */


    *m = MATRICE_VIDE;   /* FOURNI : état sûr après destruction (R6) */
}
```

3.3 Pas à pas

1.a Compléter les messages d'erreur

Dans `matrice.c`, ajoutez deux lignes `case` dans le `switch`, sur le modèle des lignes au-dessus :

```c
    case MAT_ERR_DIMENSIONS:  return "dimensions incompatibles";
    case MAT_ERR_DEPASSEMENT: return ...;   /* à vous */
```

▶ **Vérifiez :** compilez et lancez `./test_ex1` : les deux lignes `(1.a)` affichent `[OK]`.

1.b Écrire mat_est_valide

La fonction renvoie 1 si `m` est utilisable, 0 sinon. Remplacez les deux lignes provisoires par une seule instruction `return` qui combine les quatre conditions avec `&&` :

```c
    return m != NULL
        && m->coef != NULL
        && m->nb_lignes >= 1 && m->nb_lignes <= MAT_DIM_MAX
        && ...;    /* même chose pour nb_colonnes */
```

L'ordre compte : si `m` vaut `NULL`, le `&&` s'arrête avant d'évaluer `m->coef`, qui ferait planter le programme.

▶ **Vérifiez :** les quatre lignes `(1.b)` affichent `[OK]`.

1.c Refuser les dimensions invalides

Dans `mat_allouer`, à l'emplacement `(1.c)`, écrivez un `if` : si `nb_lignes < 1`, ou `nb_lignes > MAT_DIM_MAX`, ou la même chose pour `nb_colonnes`, alors `return MAT_ERR_ARGUMENT;`. Comme `*res` a déjà été mis à `MATRICE_VIDE` juste au-dessus, l'appelant reçoit bien une matrice vide.

▶ **Vérifiez :** les trois lignes `(1.c)` affichent `[OK]`.

1.d Réserver chaque ligne

Le tableau des pointeurs `coef` est déjà réservé (code fourni). Il reste à réserver les lignes. Supprimez la ligne provisoire `(void)nb_colonnes;` et écrivez la boucle indiquée dans le commentaire :

```
    for (int i = 0; i < nb_lignes; i++) {
        coef[i] = mat_calloc((size_t)nb_colonnes, sizeof *coef[i]);
        if (coef[i] == NULL) {
            /* échec : rendre ce qui a déjà été obtenu */
            ...
            return MAT_ERR_ALLOCATION;
        }
    }
```

Que faut-il rendre en cas d'échec ? Faites un dessin pour $i = 2$: les lignes 0 et 1 et le tableau `coef` ont été obtenus, la ligne 2 non. Il faut libérer les lignes `i - 1`, ..., `0`, puis `coef`. Le squelette donne la boucle à écrire.

> **Indice — la taille d'une ligne**
>
> `coef[i]` est de type `int *`, donc `*coef[i]` est un `int` : `sizeof *coef[i]` vaut `sizeof(int)`. Écrire la taille ainsi évite de se tromper de type. Le `(size_t)` convertit le nombre de colonnes dans le type attendu par `calloc`.

1.e Remplir le résultat

Tout a réussi : supprimez les deux lignes provisoires de la fin et remplacez-les par trois affectations et le retour :

```
    res->coef = coef;
    res->nb_lignes = nb_lignes;
    res->nb_colonnes = ...;
    return MAT_OK;
```

▶ **Vérifiez :** les lignes `(1.d)` et `(1.e)` affichent `[OK]`, y compris la ligne sur les échecs d'allocation simulés.

1.f Écrire mat_detruire

À l'emplacement `(1.f)` : si `m->coef` n'est pas `NULL`, une boucle `for` libère `m->coef[i]` pour `i` allant de 0 à `m->nb_lignes - 1`, *puis* on libère `m->coef`. La dernière ligne, fournie, remet la matrice à vide : un second appel ne fera donc rien.

▶ **Vérifiez :** le bilan affiche `16 / 16`, puis lancez `valgrind --leak-check=full ./test_ex1` : la fin du rapport doit contenir *All heap blocks were freed – no leaks are possible*. Si vous oubliez l'étape 1.f, tous les tests passent quand même, mais `valgrind` signale des fuites : c'est lui qui le révèle.

3.4 Compiler et tester

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g -DMAT_TEST_ALLOC \
    test_ex1.c matrice.c matrice_alloc.c -o test_ex1
./test_ex1
valgrind --leak-check=full ./test_ex1
```

Quand l'exercice est terminé, `./test_ex1` affiche :

```
=== Exercice 1 : matrice.c ===
[OK]    (1.a) message pour MAT_ERR_DIMENSIONS
[OK]    (1.a) message pour MAT_ERR_DEPASSEMENT
[OK]    (1.b) une matrice 1 x 2 est valide
[OK]    (1.b) NULL n'est pas valide
[OK]    (1.b) MATRICE_VIDE n'est pas valide
[OK]    (1.b) une matrice de 0 ligne n'est pas valide
=== Exercice 1 : matrice_alloc.c ===
[OK]    (1.c) mat_allouer refuse 0 ligne
[OK]    (1.c) mat_allouer refuse -2 colonnes
[OK]    (1.c) mat_allouer refuse une dimension > MAT_DIM_MAX
[OK]    (1.e) mat_allouer(&m, 3, 4) renvoie MAT_OK
[OK]    (1.e) m a 3 lignes et 4 colonnes
[OK]    (1.d) tous les coefficients valent 0
[OK]    (1.f) apres mat_detruire, m est vide
[OK]    (1.f) un second mat_detruire est sans danger
[OK]    (1.f) mat_detruire(NULL) ne plante pas
=== Exercice 1 : echecs d'allocation simules ===
[OK]    (1.d) chaque echec renvoie MAT_ERR_ALLOCATION et laisse *res vide

Bilan : 16 / 16 verifications reussies.
Pensez a verifier l'absence de fuite : valgrind --leak-check=full ./test_ex1
```

Erreurs fréquentes

- Réserver le tableau de lignes avec `sizeof(int)` au lieu de `sizeof(int *)` : sur une machine 64 bits, le tableau est deux fois trop petit. (Le code fourni écrit `sizeof *coef`, qui évite ce piège.)
- Libérer `coef` *avant* les lignes : les adresses des lignes sont alors lues dans une zone déjà libérée.
- Remplir `res->coef` avant la fin de la boucle : en cas d'échec, l'appelant reçoit une matrice à moitié construite.
- Oublier de tester le résultat de chaque `mat_calloc`.
- Laisser une ligne provisoire en place : relisez les commentaires `ligne provisoire` quand vous terminez le fichier.

3.5 Questions de réflexion

Q3.1 Pourquoi `mat_detruire` prend-elle un `Matrice *` et non un `Matrice` ? Que se passerait-il, pour l'appelant, avec un passage par valeur ?

Q3.2 Combien d'appels à `calloc` et à `free` faut-il pour une matrice $n \times p$? Si la k-ième allocation échoue, combien de libérations `mat_allouer` doit-elle effectuer ?

Q3.3 Appeler deux fois `free` sur la même adresse est un comportement indéfini. Pourquoi appeler deux fois `mat_detruire` sur la même matrice est-il sans danger ? Est-ce encore vrai si l'on a fait

une copie `Matrice b = a;` puis appelé `mat_detruire(&a)` et `mat_detruire(&b)` ?

Q3.4 Pourquoi `mat_allouer` commence-t-elle par l'affectation `*res = MATRICE_VIDE`, avant même de vérifier les dimensions ?

3.6 Critères de validation

- ☐ `test_ex1` compile sans aucun avertissement.
- ☐ `./test_ex1` affiche `Bilan : 16 / 16`.
- ☐ `valgrind --leak-check=full ./test_ex1` affiche *All heap blocks were freed*.
- ☐ Il ne reste aucune ligne provisoire dans `matrice.c` et `matrice_alloc.c`.

4 Exercice 2 — Entrées-sorties

Fichier à compléter : `matrice_io.c`. Objectif : afficher une matrice, l'enregistrer dans un fichier texte au format défini en partie 1, puis la relire en détectant toute donnée invalide. L'exercice 1 doit être terminé.

4.1 Ce qu'il faut comprendre

Fonction	Ce qu'il faut savoir
`fopen(chemin, mode)`	mode `"r"` (lecture) ou `"w"` (écriture ; le fichier est créé, ou *vidé* s'il existe). Renvoie `NULL` en cas d'échec : la raison est alors dans `errno` (`strerror(errno)` donne un message).
`fscanf(f, format, ...)`	renvoie le nombre d'éléments lus, ou `EOF`. Les formats `"%d"` et `"%s"` sautent d'abord les blancs (espaces, tabulations, sauts de ligne).
`fprintf(f, format, ...)`	renvoie le nombre de caractères écrits, ou un nombre négatif en cas d'erreur.
`fflush(f)`	force l'écriture du tampon ; renvoie `EOF` en cas d'erreur.
`fclose(f)`	ferme le flux ; renvoie `EOF` en cas d'erreur (en écriture, c'est souvent là qu'apparaît un disque plein).
`ferror(f)`	différencie une *erreur* du flux d'une simple *fin de fichier*.

Qui ouvre, ferme (règle R7)

`mat_lire` et `mat_ecrire` reçoivent un flux *déjà ouvert* et ne le ferment pas. C'est le code appelant, qui a fait le `fopen`, qui fait le `fclose`. Avantages : une même fonction sert pour un fichier, pour `stdout`, ou pour lire plusieurs matrices à la suite dans un même fichier.

4.2 Le module fourni

Fichier fourni `matrice_io.h`

```c
/*
 * matrice_io.h -- Lecture, écriture et affichage des matrices.
 *
 * Format texte d'une matrice :
 *   ligne 1          : <nb_lignes> <nb_colonnes>
 *   lignes suivantes : les nb_colonnes coefficients d'une ligne,
 *                      séparés par des espaces.
```

```
 *
 * Ces fonctions n'ouvrent ni ne ferment jamais de fichier : c'est le
 * code qui a appelé fopen qui doit appeler fclose.
 */
#ifndef MATRICE_IO_H
#define MATRICE_IO_H

#include <stdio.h>
#include "matrice.h"

/*
 * Lit une matrice depuis le flux f (ouvert en lecture).
 *   Préconditions : f != NULL, res != NULL.
 *   Succès : *res possède sa mémoire (à libérer avec mat_detruire) ;
 *            le flux est positionné juste après le dernier coefficient.
 *   Échec  : *res == MATRICE_VIDE (si res != NULL), aucune fuite.
 *   Erreurs : MAT_ERR_ARGUMENT, MAT_ERR_FORMAT, MAT_ERR_LECTURE,
 *             MAT_ERR_ALLOCATION.
 */
MatErreur mat_lire(FILE *f, Matrice *res);

/*
 * Écrit la matrice *m dans le flux f (ouvert en écriture) au format
 * ci-dessus. *m n'est pas modifiée.
 *   Erreurs : MAT_ERR_ARGUMENT (f NULL ou m invalide), MAT_ERR_ECRITURE.
 */
MatErreur mat_ecrire(FILE *f, const Matrice *m);

/*
 * Affiche *m sur la sortie standard, colonnes alignées, précédée de son
 * nom et de ses dimensions. Outil de mise au point : n'échoue jamais
 * (une matrice vide ou invalide est signalée comme telle).
 */
void mat_afficher(const char *nom, const Matrice *m);

#endif /* MATRICE_IO_H */
```

Le fichier de départ contient déjà la fonction interne `lire_entier`, qui lit *un* entier dans le flux en détectant les données invalides (lettres, nombre trop grand, fin de fichier). Vous n'avez pas à l'écrire : il suffit de l'appeler. Elle est `static`, donc invisible hors de ce fichier.

Fichier fourni `matrice_io.c`

```
/*
 * matrice_io.c -- Lecture, écriture et affichage des matrices.
 *
 * FICHIER À COMPLÉTER (exercice 2, étapes 2.a à 2.f).
 */
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include "matrice_io.h"
#include "matrice_alloc.h"

/* ------------------------------------------------------------------
 * FOURNI -- Ne pas modifier.
 * Lit le prochain entier du flux f et le range dans *val.
 * Renvoie MAT_OK, MAT_ERR_FORMAT (pas un entier, entier trop grand,
 * fin de fichier) ou MAT_ERR_LECTURE (erreur du flux).
 * ------------------------------------------------------------------ */
static MatErreur lire_entier(FILE *f, int *val)
```

```c
{
    char mot[32];
    char *fin;

    if (fscanf(f, "%31s", mot) != 1)
        return ferror(f) ? MAT_ERR_LECTURE : MAT_ERR_FORMAT;

    errno = 0;
    long v = strtol(mot, &fin, 10);
    if (fin == mot || *fin != '\0' || errno == ERANGE
        || v < INT_MIN || v > INT_MAX)
        return MAT_ERR_FORMAT;

    *val = (int)v;
    return MAT_OK;
}

void mat_afficher(const char *nom, const Matrice *m)
{
    if (nom == NULL)                                    /* FOURNI */
        nom = "matrice";
    if (!mat_est_valide(m)) {
        printf("%s : matrice vide ou invalide\n", nom);
        return;
    }
    printf("%s (%d x %d) :\n", nom, m->nb_lignes, m->nb_colonnes);

    /* (2.a) À COMPLÉTER : pour chaque ligne i, afficher chaque
     *       coefficient avec printf(" %8d", m->coef[i][j]),
     *       puis passer à la ligne avec putchar('\n').          */
}

MatErreur mat_ecrire(FILE *f, const Matrice *m)
{
    if (f == NULL || !mat_est_valide(m))                /* FOURNI */
        return MAT_ERR_ARGUMENT;

    /* FOURNI : première ligne du fichier = les dimensions. */
    if (fprintf(f, "%d %d\n", m->nb_lignes, m->nb_colonnes) < 0)
        return MAT_ERR_ECRITURE;

    /* (2.b) À COMPLÉTER : pour chaque ligne i de la matrice :
     *         - écrire les coefficients séparés par UN espace :
     *             le premier (j == 0) avec "%d", les autres avec " %d" ;
     *         - terminer la ligne avec fputc('\n', f) ;
     *         - si un fprintf renvoie un nombre < 0, ou si fputc
     *           renvoie EOF : return MAT_ERR_ECRITURE;          */


    /* FOURNI : forcer l'écriture du tampon et détecter une erreur. */
    if (fflush(f) == EOF)
        return MAT_ERR_ECRITURE;
    return MAT_OK;
}

MatErreur mat_lire(FILE *f, Matrice *res)
{
    /* FOURNI : contrôle des paramètres, *res mis à l'état vide. */
    if (res == NULL)
        return MAT_ERR_ARGUMENT;
    *res = MATRICE_VIDE;
    if (f == NULL)
```

```
        return MAT_ERR_ARGUMENT;

    /* FOURNI : lecture des deux dimensions. */
    int nb_lignes = 0, nb_colonnes = 0;
    MatErreur err = lire_entier(f, &nb_lignes);
    if (err == MAT_OK)
        err = lire_entier(f, &nb_colonnes);
    if (err != MAT_OK)
        return err;

    /* (2.c) À COMPLÉTER : si nb_lignes ou nb_colonnes n'est pas compris
     *       entre 1 et MAT_DIM_MAX, renvoyer MAT_ERR_FORMAT
     *       (c'est le contenu du fichier qui est faux).              */


    /* (2.d) À COMPLÉTER : déclarer une matrice LOCALE et l'allouer :
     *         Matrice tmp;
     *         err = mat_allouer(&tmp, nb_lignes, nb_colonnes);
     *         if (err != MAT_OK) return err;                          */


    /* (2.e) À COMPLÉTER : deux boucles imbriquées (i, puis j) qui lisent
     *       chaque coefficient avec lire_entier(f, &tmp.coef[i][j]).
     *       Si une lecture échoue :
     *         mat_detruire(&tmp);  return err;                        */


    /* (2.f) À COMPLÉTER : tout a réussi, on transfère le résultat :
     *         *res = tmp;  return MAT_OK;                             */

    return MAT_ERR_LECTURE;  /* ligne provisoire : à remplacer en (2.f) */
}
```

4.3 Pas à pas

On commence par l'affichage, qui servira ensuite à vérifier tout le reste, puis l'écriture, et enfin la lecture.

2.a Afficher les coefficients

Dans `mat_afficher`, le nom et les dimensions sont déjà affichés. Ajoutez deux boucles imbriquées : pour chaque ligne `i`, pour chaque colonne `j`, afficher le coefficient sur 8 caractères, puis passer à la ligne à la fin de chaque ligne de la matrice :

```
    for (int i = 0; i < m->nb_lignes; i++) {
        for (int j = 0; j < m->nb_colonnes; j++)
            printf(" %8d", m->coef[i][j]);
        putchar('\n');
    }
```

► **Vérifiez :** compilez et lancez `./test_ex2` : la matrice `M` s'affiche sur 4 lignes alignées, de 1 à 12.

2.b Écrire les coefficients dans le fichier

Dans `mat_ecrire`, la première ligne (les dimensions) est déjà écrite. À l'emplacement `(2.b)`, écrivez la même double boucle qu'en 2.a, avec trois différences :

- on écrit dans le fichier avec `fprintf(f, ...)` au lieu de `printf` ;

- format canonique : le premier coefficient d'une ligne (`j == 0`) s'écrit avec `"%d"`, les suivants avec `" %d"` (un espace devant), pour n'avoir aucun espace en fin de ligne ; on peut écrire `fprintf(f, j == 0 ? "%d" : " %d", m->coef[i][j]);`
- chaque écriture est vérifiée : si `fprintf` renvoie un nombre négatif, ou si `fputc('\n', f)` renvoie `EOF`, on fait `return MAT_ERR_ECRITURE;`.

► **Vérifiez :** les lignes `(2.b)` affichent `[OK]`. Ouvrez `test_ex2_sortie.txt` avec un éditeur : il contient `4 3`, puis `1 2 3`, `4 5 6`, `7 8 9`, `10 11 12`.

2.c Lire : vérifier les dimensions lues

Dans `mat_lire`, les deux dimensions sont déjà lues (code fourni). À l'emplacement `(2.c)`, si l'une d'elles n'est pas comprise entre 1 et `MAT_DIM_MAX`, renvoyez `MAT_ERR_FORMAT`. Ce n'est pas `MAT_ERR_ARGUMENT` : les paramètres de la fonction sont corrects, c'est le *contenu du fichier* qui est faux.

2.d Lire : réserver une matrice locale

Écrivez exactement le code indiqué en commentaire :

```
    Matrice tmp;
    err = mat_allouer(&tmp, nb_lignes, nb_colonnes);
    if (err != MAT_OK)
        return err;
```

On remplit `tmp`, une variable *locale*, et non `*res` directement : si une erreur survient au milieu du fichier, l'appelant ne recevra jamais une matrice à moitié remplie (question Q4.1).

2.e Lire : lire les coefficients

Deux boucles imbriquées (`i`, puis `j`) appellent `lire_entier(f, &tmp.coef[i][j])` et rangent le résultat dans `err`. Si `err` n'est pas `MAT_OK`, il faut d'abord *rendre la mémoire* de `tmp`, puis renvoyer l'erreur :

```
            err = lire_entier(f, &tmp.coef[i][j]);
            if (err != MAT_OK) {
                mat_detruire(&tmp);
                return err;
            }
```

2.f Lire : transférer le résultat

Remplacez la ligne provisoire par `*res = tmp;` suivi de `return MAT_OK;`. À partir de là, c'est l'appelant qui possède la matrice et qui devra la détruire (règle R2).

► **Vérifiez :** toutes les lignes de `./test_ex2` affichent `[OK]` (`Bilan : 14 / 14`), y compris les six fichiers incorrects et l'échec d'allocation simulé, et `valgrind` ne signale aucune fuite.

4.4 Compiler et tester

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g -DMAT_TEST_ALLOC \
    test_ex2.c matrice.c matrice_alloc.c matrice_io.c -o test_ex2
./test_ex2
valgrind --leak-check=full ./test_ex2
```

Quand l'exercice est terminé, `./test_ex2` affiche :

```
=== Exercice 2 : affichage (verification visuelle) ===
M (4 x 3) :
        1        2        3
        4        5        6
        7        8        9
       10       11       12
(2.a) Vous devez voir ci-dessus 4 lignes : 1 2 3 / 4 5 6 / 7 8 9 / 10 11 12
=== Exercice 2 : ecriture ===
[OK]    ouverture de test_ex2_sortie.txt en ecriture
[OK]    (2.b) mat_ecrire renvoie MAT_OK
[OK]    (2.b) le fichier contient exactement le format attendu
[OK]    (2.b) mat_ecrire refuse une matrice vide
=== Exercice 2 : relecture ===
[OK]    (2.f) mat_lire relit le fichier : MAT_OK
[OK]    (2.e) la matrice relue est identique
[OK]    (2.e) espaces et sauts de ligne en trop sont toleres
=== Exercice 2 : fichiers incorrects ===
[OK]    (2.c) fichier vide
[OK]    (2.c) dimension nulle
[OK]    (2.c) dimension negative
[OK]    (2.e) coefficient manquant
[OK]    (2.e) donnee non numerique
[OK]    (2.e) entier trop grand pour un int
=== Exercice 2 : echec d'allocation simule ===
[OK]    (2.d) mat_lire renvoie MAT_ERR_ALLOCATION proprement

Bilan : 14 / 14 verifications reussies.
Pensez a verifier l'absence de fuite : valgrind --leak-check=full ./test_ex2
```

Erreurs fréquentes

- Piloter la lecture avec `while (!feof(f))` : `feof` ne devient vrai *qu'après* une lecture ratée. Ici, on sait combien d'entiers lire : les dimensions suffisent.
- Lire les coefficients directement dans `res->coef`.
- Oublier `mat_detruire(&tmp)` quand un coefficient est invalide : `valgrind` signale alors une fuite.
- Appeler `fclose` dans `mat_lire` ou `mat_ecrire` : le fichier a été ouvert par l'appelant, c'est lui qui le ferme (règle R7).
- Écrire un espace après le dernier coefficient d'une ligne : le test sur le contenu exact du fichier échoue.

4.5 Questions de réflexion

Q4.1 Pourquoi `mat_lire` remplit-elle d'abord une matrice locale avant de l'affecter à `*res` ?

Q4.2 La lecture vérifie-t-elle que chaque ligne de la matrice est sur sa propre ligne de texte ? Est-ce

gênant ? Justifiez le principe « tolérant en lecture, strict en écriture ».

Q4.3 Après l'appel `mat_lire(f, &m)`, le flux `f` est positionné après le dernier coefficient. Comment exploiter cette propriété pour stocker deux matrices dans un même fichier ?

Q4.4 Pourquoi `mat_ecrire` appelle-t-elle `fflush` avant de renvoyer `MAT_OK` ? Cela dispense-t-il l'appelant de tester `fclose` ?

4.6 Critères de validation

- ☐ `test_ex2` compile sans aucun avertissement.
- ☐ `./test_ex2` affiche `Bilan : 14 / 14`.
- ☐ `valgrind --leak-check=full ./test_ex2` affiche *All heap blocks were freed*.
- ☐ Aucune fonction de `matrice_io.c` n'appelle `fopen` ni `fclose`, et il ne reste aucune ligne provisoire.

5 Exercice 3 — Calcul matriciel

Fichier à compléter : `matrice_calcul.c`. Objectif : additionner et multiplier deux matrices en vérifiant leurs dimensions, sans jamais modifier les matrices d'entrée, et en produisant un résultat qui possède sa propre mémoire. L'exercice 1 doit être terminé (l'exercice 2 n'est pas nécessaire).

5.1 L'addition

Deux matrices ne peuvent être additionnées que si elles ont **exactement les mêmes dimensions** $n \times p$. Le résultat est alors de dimensions $n \times p$, et $c_{ij} = a_{ij} + b_{ij}$:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{pmatrix} = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix} \qquad (2 \times 3) + (2 \times 3) = (2 \times 3).$$

Une matrice 2×3 et une matrice 3×2 ont le même nombre de coefficients, mais ne s'additionnent *pas*.

5.2 La multiplication

Le produit $C = A \times B$ n'est défini que si **le nombre de colonnes de A est égal au nombre de lignes de B**. Si A est $n \times p$ et B est $p \times q$, alors C est $n \times q$ et

$$c_{ij} \;=\; \sum_{k=1}^{p} a_{ik}\, b_{kj} \qquad \text{(ligne } i \text{ de } A \text{ « multipliée » par la colonne } j \text{ de } B\text{)}.$$

Exemple.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix} = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix} \qquad c_{11} = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 58.$$

Le produit n'est pas commutatif : avec les mêmes matrices, $B \times A$ est une matrice 3×3, et $A \times A$ n'est pas défini (2×3 par 2×3).

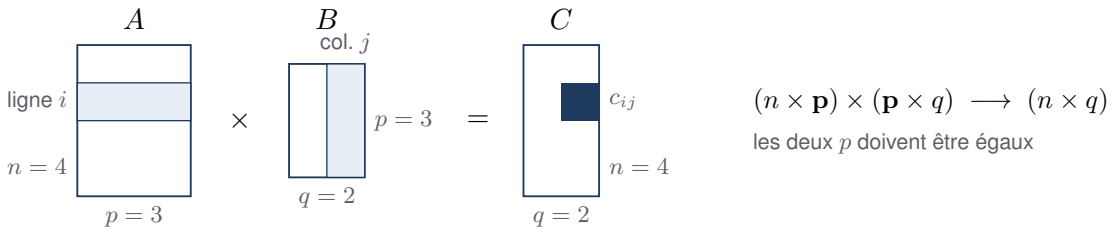


Figure 5: Règle des dimensions du produit et calcul d'un coefficient.

5.3 Le dépassement de capacité

Un `int` ne peut contenir que des valeurs comprises entre `INT_MIN` et `INT_MAX` (souvent -2^{31} et $2^{31} - 1$, soit environ $\pm 2{,}1$ milliards). Le dépassement d'un calcul sur des `int` signés est un *comportement indéfini* en C : il faut donc le détecter *avant* qu'il ne se produise. On calcule pour cela dans un type plus large, `long long` (au moins 64 bits) :

- la somme de deux `int`, ou le produit de deux `int`, tient toujours dans un `long long` ;
- on vérifie ensuite que le résultat final est entre `INT_MIN` et `INT_MAX`, sinon on renvoie `MAT_ERR_DEPASSEMENT`.

Pour le produit, la somme de p produits peut elle-même dépasser la capacité d'un `long long` : il faut donc aussi vérifier chaque addition à la somme. Le code de ce contrôle vous est donné à l'étape 3.g.

5.4 Le module fourni

Fichier fourni `matrice_calcul.h`

```c
/*
 * matrice_calcul.h -- Opérations arithmétiques sur les matrices.
 *
 * Règles communes aux deux fonctions :
 *   - a et b ne sont jamais modifiées (paramètres const) ;
 *   - res est un paramètre de SORTIE : son contenu antérieur est ignoré
 *     et écrasé ; s'il possédait de la mémoire, l'appelant doit l'avoir
 *     libérée avant l'appel ;
 *   - res doit être distinct de a et de b (sinon MAT_ERR_ARGUMENT, et
 *     rien n'est modifié) ;
 *   - en cas de succès, *res possède sa propre mémoire (mat_detruire) ;
 *   - en cas d'échec, *res == MATRICE_VIDE (si res != NULL), aucune fuite.
 */
#ifndef MATRICE_CALCUL_H
#define MATRICE_CALCUL_H

#include "matrice.h"

/*
 * res = a + b.
 *   Condition : mêmes dimensions (a et b de taille n x p) ; res : n x p.
 *   Erreurs : MAT_ERR_ARGUMENT, MAT_ERR_DIMENSIONS, MAT_ERR_ALLOCATION,
 *             MAT_ERR_DEPASSEMENT.
 */
MatErreur mat_additionner(const Matrice *a, const Matrice *b,
                          Matrice *res);

/*
```

```
 * res = a x b (produit matriciel).
 *   Condition : a de taille n x p, b de taille p x q ; res : n x q.
 *   Erreurs : MAT_ERR_ARGUMENT, MAT_ERR_DIMENSIONS, MAT_ERR_ALLOCATION,
 *             MAT_ERR_DEPASSEMENT.
 */
MatErreur mat_multiplier(const Matrice *a, const Matrice *b,
                         Matrice *res);

#endif /* MATRICE_CALCUL_H */
```

Le fichier de départ contient deux fonctions internes déjà écrites : `verifier_parametres`, qui fait les contrôles communs aux deux opérations (et met `*res` à l'état vide), et `tient_dans_int`, qui dit si un résultat calculé en `long long` tient dans un `int`. La structure des boucles est aussi fournie : vous complétez les calculs.

Fichier fourni `matrice_calcul.c`

```c
/*
 * matrice_calcul.c -- Opérations arithmétiques sur les matrices.
 *
 * FICHIER À COMPLÉTER (exercice 3, étapes 3.a à 3.g).
 */
#include <limits.h>
#include <stddef.h>
#include "matrice_calcul.h"
#include "matrice_alloc.h"

/* ---------------------------------------------------------------------
 * FOURNI -- Contrôles communs aux deux opérations.
 * Refuse res NULL ou égal à une entrée (sans rien modifier), met *res
 * à l'état vide, puis vérifie que a et b sont valides.
 * --------------------------------------------------------------------- */
static MatErreur verifier_parametres(const Matrice *a, const Matrice *b,
                                     Matrice *res)
{
    if (res == NULL || res == a || res == b)
        return MAT_ERR_ARGUMENT;
    *res = MATRICE_VIDE;
    if (!mat_est_valide(a) || !mat_est_valide(b))
        return MAT_ERR_ARGUMENT;
    return MAT_OK;
}

/* FOURNI -- Un résultat calculé en long long tient-il dans un int ? */
static int tient_dans_int(long long v)
{
    return v >= INT_MIN && v <= INT_MAX;
}

MatErreur mat_additionner(const Matrice *a, const Matrice *b,
                          Matrice *res)
{
    MatErreur err = verifier_parametres(a, b, res);      /* FOURNI */
    if (err != MAT_OK)
        return err;

    /* (3.a) À COMPLÉTER : si a et b n'ont pas le même nombre de lignes
     *       OU pas le même nombre de colonnes,
     *       renvoyer MAT_ERR_DIMENSIONS.                          */


    /* (3.b) À COMPLÉTER : allouer r avec les dimensions de a ;
```

```c
     *         err = mat_allouer(&r, ..., ...);
     *         if (err != MAT_OK) return err;                          */
    Matrice r = MATRICE_VIDE;


    /* FOURNI : parcours de tous les coefficients de r. */
    for (int i = 0; i < r.nb_lignes; i++) {
        for (int j = 0; j < r.nb_colonnes; j++) {
            /* (3.c) À COMPLÉTER : calculer la somme des coefficients
             *       (i, j) de a et de b EN long long :
             *         long long s = (long long)a->coef[i][j] + ... ;  */
            long long s = 0;   /* ligne provisoire : à remplacer en (3.c) */

            if (!tient_dans_int(s)) {                   /* FOURNI */
                mat_detruire(&r);
                return MAT_ERR_DEPASSEMENT;
            }
            r.coef[i][j] = (int)s;
        }
    }

    /* (3.d) À COMPLÉTER : transférer le résultat :
     *         *res = r;  return MAT_OK;                               */
    mat_detruire(&r);            /* ligne provisoire : à supprimer en (3.d) */
    return MAT_ERR_LECTURE;      /* ligne provisoire : à remplacer en (3.d) */
}

MatErreur mat_multiplier(const Matrice *a, const Matrice *b,
                         Matrice *res)
{
    MatErreur err = verifier_parametres(a, b, res);      /* FOURNI */
    if (err != MAT_OK)
        return err;

    /* (3.e) À COMPLÉTER : le produit a x b n'existe que si le nombre de
     *       COLONNES de a est égal au nombre de LIGNES de b.
     *       Sinon, renvoyer MAT_ERR_DIMENSIONS.                       */


    /* (3.f) À COMPLÉTER : allouer r de dimensions
     *       (lignes de a) x (colonnes de b).                          */
    Matrice r = MATRICE_VIDE;


    for (int i = 0; i < r.nb_lignes; i++) {              /* FOURNI */
        for (int j = 0; j < r.nb_colonnes; j++) {
            long long s = 0;

            /* (3.g) À COMPLÉTER : boucle sur k, de 0 à a->nb_colonnes - 1,
             *       qui ajoute à s le produit a(i,k) x b(k,j) :
             *
             *   for (int k = 0; k < a->nb_colonnes; k++) {
             *       long long p = (long long)a->coef[i][k] * b->coef[k][j];
             *       if ((p > 0 && s > LLONG_MAX - p)
             *           || (p < 0 && s < LLONG_MIN - p)) {
             *           mat_detruire(&r);
             *           return MAT_ERR_DEPASSEMENT;
             *       }
             *       s += p;
             *   }                                                     */

            if (!tient_dans_int(s)) {                   /* FOURNI */
```

```
                mat_detruire(&r);
                return MAT_ERR_DEPASSEMENT;
            }
            r.coef[i][j] = (int)s;
        }
    }

    /* (3.h) À COMPLÉTER : *res = r;  return MAT_OK;                    */
    mat_detruire(&r);            /* ligne provisoire : à supprimer en (3.h) */
    return MAT_ERR_LECTURE;      /* ligne provisoire : à remplacer en (3.h) */
}
```

5.5 Pas à pas

3.a Addition : contrôler les dimensions

À l'emplacement `(3.a)`, écrivez un `if` qui compare `a->nb_lignes` à `b->nb_lignes` *et* `a->nb_colonnes` à `b->nb_colonnes`. Si l'une des deux comparaisons est fausse, renvoyez `MAT_ERR_DIMENSIONS`.

3.b Addition : réserver le résultat

La variable `r` est déjà déclarée et vide. Juste après sa déclaration, réservez-la avec les dimensions de `a` :

```
    err = mat_allouer(&r, a->nb_lignes, a->nb_colonnes);
    if (err != MAT_OK)
        return err;
```

Tant que `r` est vide (0 ligne), les boucles fournies ne font rien : c'est pour cela que le fichier compile et s'exécute avant cette étape.

3.c Addition : calculer chaque coefficient

Remplacez la ligne provisoire `long long s = 0;` par la somme calculée en `long long` :

```
            long long s = (long long)a->coef[i][j] + b->coef[i][j];
```

La conversion `(long long)` est indispensable : sans elle, la somme serait calculée en `int` et pourrait déborder *avant* le test. Le test de dépassement qui suit est fourni.

3.d Addition : transférer le résultat

Remplacez les deux lignes provisoires de la fin par `*res = r;` et `return MAT_OK;`.

► **Vérifiez :** compilez et lancez `./test_ex3` : toutes les lignes de la partie « addition » affichent `[OK]`.

3.e Multiplication : contrôler les dimensions

Le produit $a \times b$ n'existe que si le nombre de *colonnes* de `a` est égal au nombre de *lignes* de `b`. Sinon, renvoyez `MAT_ERR_DIMENSIONS`.

3.f Multiplication : réserver le résultat

Comme en 3.b, mais avec les dimensions du produit : `a->nb_lignes` lignes et `b->nb_colonnes` colonnes (revoir la figure 5).

3.g Multiplication : la somme des produits

Pour chaque coefficient (i, j), la variable `s` est initialisée à 0 (code fourni). Écrivez la boucle sur `k` qui lui ajoute $a_{ik} \times b_{kj}$; le commentaire du squelette en donne le code complet, avec le contrôle de dépassement :

```
            for (int k = 0; k < a->nb_colonnes; k++) {
                long long p = (long long)a->coef[i][k] * b->coef[k][j];
                if ((p > 0 && s > LLONG_MAX - p)
                    || (p < 0 && s < LLONG_MIN - p)) {
                    mat_detruire(&r);
                    return MAT_ERR_DEPASSEMENT;
                }
                s += p;
            }
```

Lisez la condition du `if` : elle vérifie, *avant* l'addition, que `s + p` ne dépassera pas la capacité d'un `long long`. Remarquez aussi le `mat_detruire(&r)` avant chaque `return` d'erreur : `r` a été réservée, il faut la rendre.

3.h Multiplication : transférer le résultat

Comme en 3.d.

▶ **Vérifiez :** `./test_ex3` affiche `Bilan : 14 / 14` et `valgrind` ne signale aucune fuite.

> **Indice — vérifier un calcul à la main**
>
> Le test calcule $a \times c$ avec $a = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ et $c = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$. Si le résultat est faux, ajoutez provisoirement un `printf` dans la boucle pour afficher `i`, `j`, `k` et `p`, et comparez avec le calcul de $c_{11} = 58$ fait en section 5.

5.6 Compiler et tester

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g -DMAT_TEST_ALLOC \
    test_ex3.c matrice.c matrice_alloc.c matrice_calcul.c -o test_ex3
./test_ex3
valgrind --leak-check=full ./test_ex3
```

Quand l'exercice est terminé, `./test_ex3` affiche :

```
=== Exercice 3 : addition ===
[OK]    (3.a) (2 x 3) + (3 x 2) : MAT_ERR_DIMENSIONS
[OK]    (3.d) (2 x 3) + (2 x 3) : MAT_OK
[OK]    (3.c) a + b est correct
[OK]    a et b n'ont pas ete modifiees
[OK]    (3.b) le resultat possede sa propre memoire
[OK]    (3.c) INT_MAX + 1 : MAT_ERR_DEPASSEMENT
=== Exercice 3 : multiplication ===
[OK]    (3.e) (2 x 3) x (2 x 3) : MAT_ERR_DIMENSIONS
```

```
[OK]    (3.h) (2 x 3) x (3 x 2) : MAT_OK
[OK]    (3.g) a x c = (58 64 / 139 154)
[OK]    (3.f) (3 x 2) x (2 x 3) donne une matrice 3 x 3
[OK]    a et c n'ont pas ete modifiees
[OK]    (3.g) 2 x 10^10 depasse INT_MAX : MAT_ERR_DEPASSEMENT
=== Exercice 3 : cas particuliers ===
[OK]    resultat = entree refuse, a reste intacte
[OK]    (3.f) echec d'allocation : MAT_ERR_ALLOCATION, r vide

Bilan : 14 / 14 verifications reussies.
Pensez a verifier l'absence de fuite : valgrind --leak-check=full ./test_ex3
```

Erreurs fréquentes

- Comparer `nb_lignes * nb_colonnes` pour l'addition : une matrice 2×3 et une 3×2 ont autant de coefficients mais ne s'additionnent pas.
- Se tromper de borne : dans le produit, `k` va de 0 à `a->nb_colonnes - 1` (qui vaut aussi `b->nb_lignes - 1`).
- Oublier la conversion `(long long)` : le calcul déborde en `int` avant d'être testé.
- Écrire `res->coef = a->coef;` : le résultat partagerait la mémoire de `a`, et le détruire détruirait `a`.
- Oublier `mat_detruire(&r)` avant un `return` d'erreur.

5.7 Questions de réflexion

Q5.1 Le paramètre `const Matrice *a` empêche-t-il d'écrire `a->coef[0][0] = 7;` ? Que protège exactement ce `const` ? Que faudrait-il changer pour que le compilateur refuse cette affectation ?

Q5.2 Pourquoi refuse-t-on l'appel `mat_additionner(&a, &b, &a)` alors que, mathématiquement, $a \leftarrow a + b$ a un sens ? Que se passerait-il avec notre implémentation si on l'acceptait ?

Q5.3 Combien de multiplications d'entiers effectue `mat_multiplier` pour des matrices $n \times p$ et $p \times q$? Combien pour deux matrices 1000×1000 ?

Q5.4 Pourquoi ne suffit-il pas de calculer `int s = a + b;` puis de tester `s < a` pour détecter un dépassement ?

5.8 Critères de validation

- ☐ `test_ex3` compile sans aucun avertissement.
- ☐ `./test_ex3` affiche `Bilan : 14 / 14`.
- ☐ `valgrind --leak-check=full ./test_ex3` affiche *All heap blocks were freed*.
- ☐ Il ne reste aucune ligne provisoire dans `matrice_calcul.c`.

6 Exercice 4 — Démonstration complète

Fichier à compléter : `main.c`. Objectif : assembler les modules dans un programme qui lit deux matrices, calcule, affiche et enregistre le résultat, et qui traite proprement chaque erreur. Les exercices 1 à 3 doivent être terminés.

6.1 Spécification du programme

Le programme `demo_matrices` s'utilise ainsi :

```
./demo_matrices <add|mul> <fichier_A> <fichier_B> <fichier_R>
```

Il doit, dans l'ordre :

1. vérifier les arguments (nombre, opération `add` ou `mul`) et afficher l'usage sinon ;
2. ouvrir `fichier_A`, lire la matrice A, fermer le fichier ;
3. faire de même pour B ;
4. afficher A et B avec `mat_afficher` ;
5. calculer $R = A + B$ ou $R = A \times B$;
6. afficher R ;
7. enregistrer R dans `fichier_R` (ouverture, écriture, fermeture vérifiée) ;
8. libérer toutes les matrices, *quel que soit* le chemin suivi ;
9. en cas d'erreur, afficher sur `stderr` un message précis (quel fichier, quelle opération, quelle cause) et terminer avec le code `EXIT_FAILURE` ; sinon terminer avec `EXIT_SUCCESS`.

La figure 6 résume le déroulement : toutes les erreurs convergent vers un *unique* point de nettoyage.

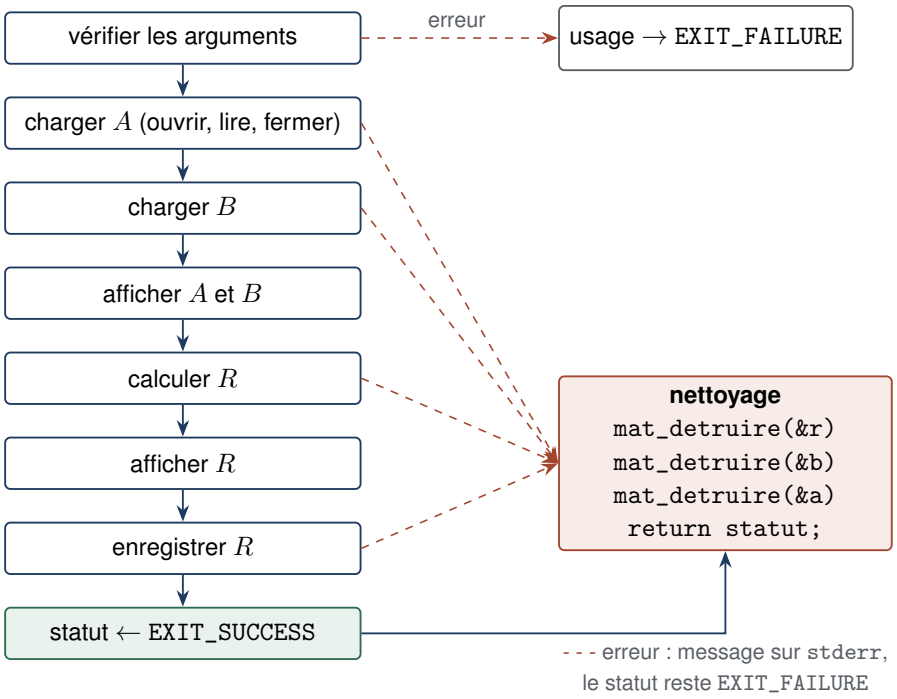


Figure 6: Déroulement du programme de démonstration : un seul point de sortie, où chaque matrice est détruite exactement une fois.

Un seul point de nettoyage

Déclarez les trois matrices au début de `main`, toutes initialisées à `MATRICE_VIDE` (R1), ainsi qu'une variable un entier `statut` initialisé à `EXIT_FAILURE`. À chaque erreur, sautez au nettoyage final avec l'instruction `goto nettoyage`. Comme `mat_detruire` est sans effet sur une matrice vide (R6), le nettoyage n'a pas besoin de savoir quelles matrices ont été créées.

> C'est l'un des rares usages de `goto` considérés comme de bonne pratique en C ; une alternative consiste à imbriquer des `if`, au prix d'une indentation profonde.

6.2 Le fichier de départ

Les deux fonctions qui manipulent les fichiers, `charger` et `sauvegarder`, sont fournies : lisez-les, elles montrent comment ouvrir un fichier, tester l'ouverture, appeler la bibliothèque et refermer le fichier. Dans `main`, la vérification des arguments, le chargement, l'enregistrement et le nettoyage final sont aussi fournis.

Fichier fourni `main.c`

```c
/*
 * main.c -- Programme de démonstration de la bibliothèque de matrices.
 *
 * FICHIER À COMPLÉTER (exercice 4, étapes 4.a à 4.d).
 *
 * Usage : ./demo_matrices <add|mul> <fichier_A> <fichier_B> <fichier_R>
 */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "matrice.h"
#include "matrice_alloc.h"
#include "matrice_io.h"
#include "matrice_calcul.h"

/* ---------------------------------------------------------------------
 * FOURNI -- Ouvre le fichier, lit une matrice, ferme le fichier.
 * Affiche un message précis sur stderr en cas d'erreur.
 * --------------------------------------------------------------------- */
static MatErreur charger(const char *chemin, Matrice *m)
{
    FILE *f = fopen(chemin, "r");
    if (f == NULL) {
        fprintf(stderr, "Erreur : impossible d'ouvrir \"%s\" : %s\n",
                chemin, strerror(errno));
        *m = MATRICE_VIDE;
        return MAT_ERR_LECTURE;
    }
    MatErreur err = mat_lire(f, m);
    fclose(f);
    if (err != MAT_OK)
        fprintf(stderr, "Erreur : lecture de \"%s\" : %s\n",
                chemin, mat_message_erreur(err));
    return err;
}

/* ---------------------------------------------------------------------
 * FOURNI -- Ouvre le fichier en écriture, écrit la matrice, ferme le
 * fichier (en testant aussi le résultat de fclose).
 * --------------------------------------------------------------------- */
static MatErreur sauvegarder(const char *chemin, const Matrice *m)
{
    FILE *f = fopen(chemin, "w");
    if (f == NULL) {
        fprintf(stderr, "Erreur : impossible de créer \"%s\" : %s\n",
                chemin, strerror(errno));
```

```c
        return MAT_ERR_ECRITURE;
    }
    MatErreur err = mat_ecrire(f, m);
    if (fclose(f) == EOF && err == MAT_OK)
        err = MAT_ERR_ECRITURE;
    if (err != MAT_OK)
        fprintf(stderr, "Erreur : écriture de \"%s\" : %s\n",
                chemin, mat_message_erreur(err));
    return err;
}

int main(int argc, char *argv[])
{
    /* FOURNI : vérification des arguments. */
    if (argc != 5
        || (strcmp(argv[1], "add") != 0 && strcmp(argv[1], "mul") != 0)) {
        fprintf(stderr, "Usage : %s <add|mul> <fichier_A> <fichier_B> "
                        "<fichier_R>\n", argv[0]);
        return EXIT_FAILURE;
    }
    int addition = (strcmp(argv[1], "add") == 0);

    /* FOURNI : toutes les matrices démarrent vides (règle R1) et le
     * programme est en échec tant qu'il n'a pas tout réussi. */
    Matrice a = MATRICE_VIDE, b = MATRICE_VIDE, r = MATRICE_VIDE;
    int statut = EXIT_FAILURE;

    /* FOURNI : chargement des deux matrices. */
    if (charger(argv[2], &a) != MAT_OK || charger(argv[3], &b) != MAT_OK)
        goto nettoyage;

    /* (4.a) À COMPLÉTER : afficher A et B avec mat_afficher.          */


    /* (4.b) À COMPLÉTER : calculer r.
     *         MatErreur err = addition ? mat_additionner(&a, &b, &r)
     *                                  : mat_multiplier(&a, &b, &r);
     *       Si err != MAT_OK : afficher sur stderr un message utilisant
     *       mat_message_erreur(err) et les dimensions de A et de B,
     *       puis : goto nettoyage;                                    */

    (void)addition;   /* ligne provisoire : à supprimer en (4.b) */

    /* (4.c) À COMPLÉTER : afficher r (nom "R = A + B" ou "R = A x B").  */


    /* FOURNI : enregistrement de r dans le fichier argv[4]. */
    if (sauvegarder(argv[4], &r) != MAT_OK)
        goto nettoyage;

    /* (4.d) À COMPLÉTER : tout a réussi :
     *         - afficher "Résultat enregistré dans \"...\"." ;
     *         - statut = EXIT_SUCCESS;                                */


nettoyage:
    /* FOURNI : point de sortie unique. Chaque matrice est détruite
     * exactement une fois, quel que soit le chemin suivi. */
    mat_detruire(&r);
    mat_detruire(&b);
    mat_detruire(&a);
    return statut;
```

```
}
```

6.3 Pas à pas

4.a Afficher A et B

Deux appels à `mat_afficher`, avec les noms `"A"` et `"B"` : `mat_afficher("A", &a);`

4.b Calculer le résultat

Supprimez la ligne provisoire `(void)addition;`. La variable `addition` vaut 1 si l'utilisateur a demandé `add`, 0 sinon. L'opérateur `? :` choisit la fonction à appeler :

```
    MatErreur err = addition ? mat_additionner(&a, &b, &r)
                             : mat_multiplier(&a, &b, &r);
    if (err != MAT_OK) {
        fprintf(stderr, "Erreur : calcul impossible : %s ",
                mat_message_erreur(err));
        fprintf(stderr, "(A : %d x %d, B : %d x %d)\n", ...);
        goto nettoyage;
    }
```

Complétez le second `fprintf` avec les dimensions de `a` et de `b`. L'instruction `goto nettoyage;` saute directement à l'étiquette `nettoyage:` en bas de `main`.

4.c Afficher le résultat

Un appel à `mat_afficher` avec le nom `"R = A + B"` ou `"R = A x B"` selon `addition` (là encore, l'opérateur `? :` est pratique).

4.d Signaler le succès

Après l'enregistrement (fourni), affichez `printf("Résultat enregistré dans \"%s\".\n", argv[4]);` puis écrivez `statut = EXIT_SUCCESS;`. Comme `statut` valait `EXIT_FAILURE` depuis le début, toute sortie par `goto nettoyage` avant cette ligne signale un échec.

▶ **Vérifiez :** le scénario 1 ci-dessous produit exactement l'affichage attendu.

6.4 Compiler et lancer

Le programme utilise les quatre modules : on compile les cinq fichiers `.c` ensemble (sans `-DMAT_TEST_ALLOC` : ce n'est pas un test).

```
gcc -std=c11 -Wall -Wextra -Wpedantic -g \
    main.c matrice.c matrice_alloc.c matrice_io.c matrice_calcul.c \
    -o demo_matrices
./demo_matrices add donnees/A.txt donnees/B.txt R.txt
```

Affichage attendu pour ce scénario 1 :

```
A (4 x 3) :
       25       48       84
       15       33       11
       45       88       10
        0       12       -5
B (4 x 3) :
```

```
         1        2        3
         4        5        6
         7        8        9
        10       11       12
R = A + B (4 x 3) :
        26       50       87
        19       38       17
        52       96       19
        10       23        7
Résultat enregistré dans "resultat_add.txt".
Code de sortie : 0
```

La dernière ligne n'est pas affichée par le programme : la commande `echo $?`, tapée juste après, affiche le code de sortie (ici 0). Le fichier `R.txt` doit contenir :

```
4 3
26 50 87
19 38 17
52 96 19
10 23 7
```

6.5 Scénarios de test

Les fichiers `A.txt`, `B.txt`, `C.txt`, `invalide.txt` et `tronque.txt` sont fournis dans le dossier `donnees/` : dans les commandes, écrivez par exemple `donnees/A.txt`.

#	Scénario	Comportement attendu	Sortie
1	`add A.txt B.txt R.txt`	affiche A, B, R ; `R.txt` créé au format canonique	0
2	`mul A.txt C.txt R.txt`	idem, R de dimensions 4×2	0
3	`add A.txt C.txt R.txt`	« dimensions incompatibles », avec les dimensions ; pas de fichier créé	1
4	`mul A.txt B.txt R.txt`	idem (4×3 par 4×3)	1
5	fichier `absent.txt`	« impossible d'ouvrir » + cause système	1
6	`invalide.txt` comme A	« fichier mal formé », avec le nom du fichier	1
7	`tronque.txt` comme A	« fichier mal formé »	1
8	résultat dans un dossier inexistant	« impossible de créer » ; matrices libérées	1
9	échec d'allocation (simulé dans `test_ex2` et `test_ex3`)	code `MAT_ERR_ALLOCATION`, aucune fuite	---
10	arguments manquants ou opération inconnue	message d'usage	1

Pour chaque scénario, vérifiez le code de sortie avec `echo $?` (sous Linux ou macOS) et, sous `valgrind`, la ligne *All heap blocks were freed – no leaks are possible*.

6.6 Questions de réflexion

Q6.1 Pourquoi le message d'erreur est-il affiché par `main.c` (ou ses fonctions `static`) et non par `mat_lire` ? Pensez à un programme graphique qui réutiliserait la bibliothèque.

Q6.2 Pourquoi le résultat de `fclose` est-il testé dans `sauvegarder` mais pas dans `charger` ?

Q6.3 Dans le scénario 8, le calcul a réussi mais l'enregistrement a échoué. Combien de matrices faut-il libérer ? Vérifiez que votre programme le fait.

6.7 Critères de validation finaux

- ☐ Le programme compile sans aucun avertissement avec `-std=c11 -Wall -Wextra -Wpedantic`.
- ☐ `main.c` n'inclut que des en-têtes ; aucun module ne contient de fonction `main`.
- ☐ Les dix scénarios donnent le comportement attendu.
- ☐ Le fichier produit par le scénario 1 peut être relu par le programme lui-même : `./demo_matrices add R.txt R.txt R2.txt`.
- ☐ `valgrind` ne signale ni fuite ni accès invalide dans *aucun* scénario.
- ☐ Les trois programmes `test_ex1`, `test_ex2`, `test_ex3` affichent toujours un bilan complet.